St Francis Institute of Technology

(Autonomous Institute)

Department of Artificial Intelligence and Machine Learning

Second Year AIML Engineering (SEM-IV A.Y. 2025-26)

Web Programming Lab. Experiment Report

Experiment 11: Database Integration using MongoDB

---

1. AIM

To integrate a MongoDB database with a Node.js and Express application, store user data, retrieve users using REST API calls, and display the data dynamically in a React frontend.

2. Lab Objectives

2.1. To understand NoSQL databases

2.2. To learn MongoDB fundamentals

2.3. To understand Mongoose and its role

2.4. To understand REST API concepts

2.5. To connect MongoDB with Express backend

2.6. To store and retrieve user data

2.7. To display backend data in React frontend

3. Lab Outcomes

After completing this experiment, students will be able to:

- Explain the difference between SQL and NoSQL
- Create a MongoDB database
- Define schemas using Mongoose
- Perform CRUD operations
- Build REST APIs
- Connect frontend React app with backend API
- Display dynamic data from database

4. Prerequisites

Students should know:

- JavaScript basics
- Node.js and Express fundamentals
- Basic React concepts
- REST API basics

5. Theory

5.1. What is NoSQL?

NoSQL stands for “Not Only SQL”.

Characteristics:

- Non-relational database
- Flexible schema
- Stores data in:
- Document format
- Key-value pairs
- Graphs
- Wide-column format

Advantages:

- High scalability
- Flexible data structure
- Good for large-scale applications

5.2. Introduction to MongoDB MongoDB is:

- A NoSQL document database
- Stores data in JSON-like format (BSON)
- Uses collections instead of tables
- Uses documents instead of rows

Key Terms:

Database → Container for collections
Collection → Group of documents
Document → JSON-like data record

5.3. Mongoose Overview

Mongoose is:• An Object Data Modeling (ODM) library. Used to interact with MongoDB in Node.js

Features:

- Schema definition
- Data validation
- Middleware support
- Query building

5.4. REST API Concept

REST (Representational State Transfer):

- Architectural style for web services
- Uses HTTP methods:

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Delete data

6. Experiment Architecture

React Frontend
⬇
Express Backend
⬇
MongoDB Database

7. Procedure

7.1. Install MongoDB

1. Install MongoDB locally OR use MongoDB Atlas.
2. Verify installation.
3. Create a new database.

```
test> use practiceDB
switched to db practiceDB
```

7.2. Setup Backend Project

1. Create a backend project folder.
2. Initialize npm.
3. Install required packages:
    a. express
    b. mongoose
    c. cors
4. Create a server file.

7.3. Connect Express to MongoDB

1. Import mongoose.

2. Provide connection string.
3. Establish database connection.
4. Confirm successful connection in terminal.

```
C:\Users\Angel\web11>node index.js
MongoDB Connected Successfully
```

7.4. Create User Schema using Mongoose

1. Define user schema:
   i. name
   ii. email
   iii. age
2. Create a User model.
3. Export model.

7.5. Create REST API Routes

1. Create User (POST)
   a. Accept user data.
   b. Save to database.

```
app.post('/users', async (req, res) => {
  try {
    const user = new User(req.body); // accept da
    await user.save();               // save to DB
    res.status(201).json(user);      // send JSON
response
  } catch (err) {
    res.status(400).json({ error: err.message });
  }});
```

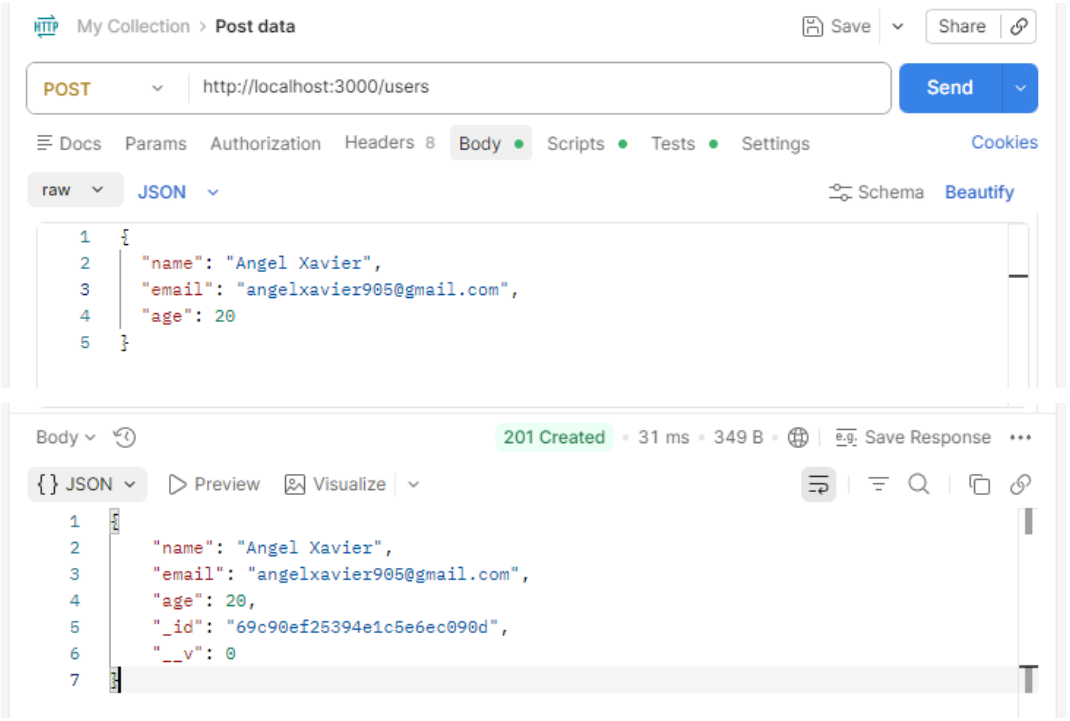
2. Retrieve All Users (GET)
   a. Fetch all users from database.
   b. Send JSON response.

```
app.get('/users', async (req, res) => {
  try {    const users = await User.find(); // fetch a
users
    res.status(200).json(users);     // send JSON
array
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

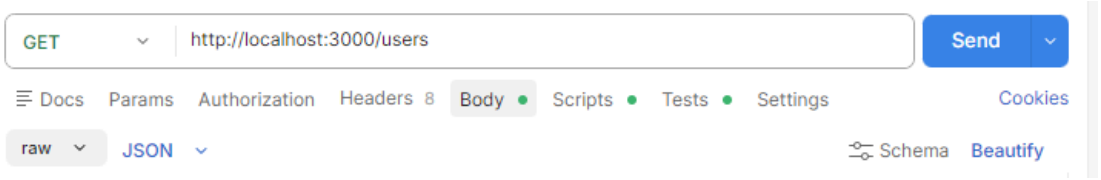
3. Retrieve Single User (GET by ID)
   a. Fetch user by ID.

7.6. Test API using Postman

1. Send POST request to create user.



2. Send GET request to retrieve users.



3. Verify JSON response.

4. Check data stored in MongoDB.

7.7. Setup React Frontend

1. Create React app.
2. Create component:
    a. UserList
3. Use fetch/axios to call backend GET API.
4. Store data in state.
5. Display user data using:
    a. Table OR Cards.

7.8. Display Data Dynamically

1. Use map() to iterate through users.
2. Render each user dynamically.
3. Verify UI updates when new user is added.

7.9. Verify Full Integration

1. Add user via Postman.
2. Refresh React frontend.
3. Confirm data appears dynamically.

8. Expected Results / Outcome

8.1. Was the MongoDB database created successfully and visible in MongoDB Compass/Atlas?

8.2. Is the Express backend successfully connected to the MongoDB database without connection errors?

8.3. When a POST request is made, is the user data stored correctly in the database collection?

8.4. Do the implemented REST API endpoints return data in proper JSON format?

8.5. Is the React frontend able to fetch user data successfully from the backend API?

8.6. Does the user list display dynamically on the frontend when data is retrieved from the database?

8.7. Does the stored data remain available even after restarting the server?

9. Conclusion Questions

1. What is Mongoose used for?
2. What is a schema?
3. What is REST API?
4. How does frontend communicate with backend?

10. Post-Experiment Questions

1. What happens if MongoDB connection fails?
2. What is the difference between collection and document?
3. Why is schema validation important?
4. How can you update or delete user data?
5. What is the role of CORS?

11. References

1. **MongoDB: The Definitive Guide** –

Kristina Chodorow, O'Reilly Media.

2. **Web Development with Node and Express** – Ethan Brown, O'Reilly Media.

3. Official MongoDB Documentation –

MongoDB

https://www.mongodb.com/docs/

4. Mongoose Documentation –

Mongoose

https://mongoosejs.com/docs/